Ficha 8 (26) - Formulários

Ficha 8 - Portfólio com Formulários

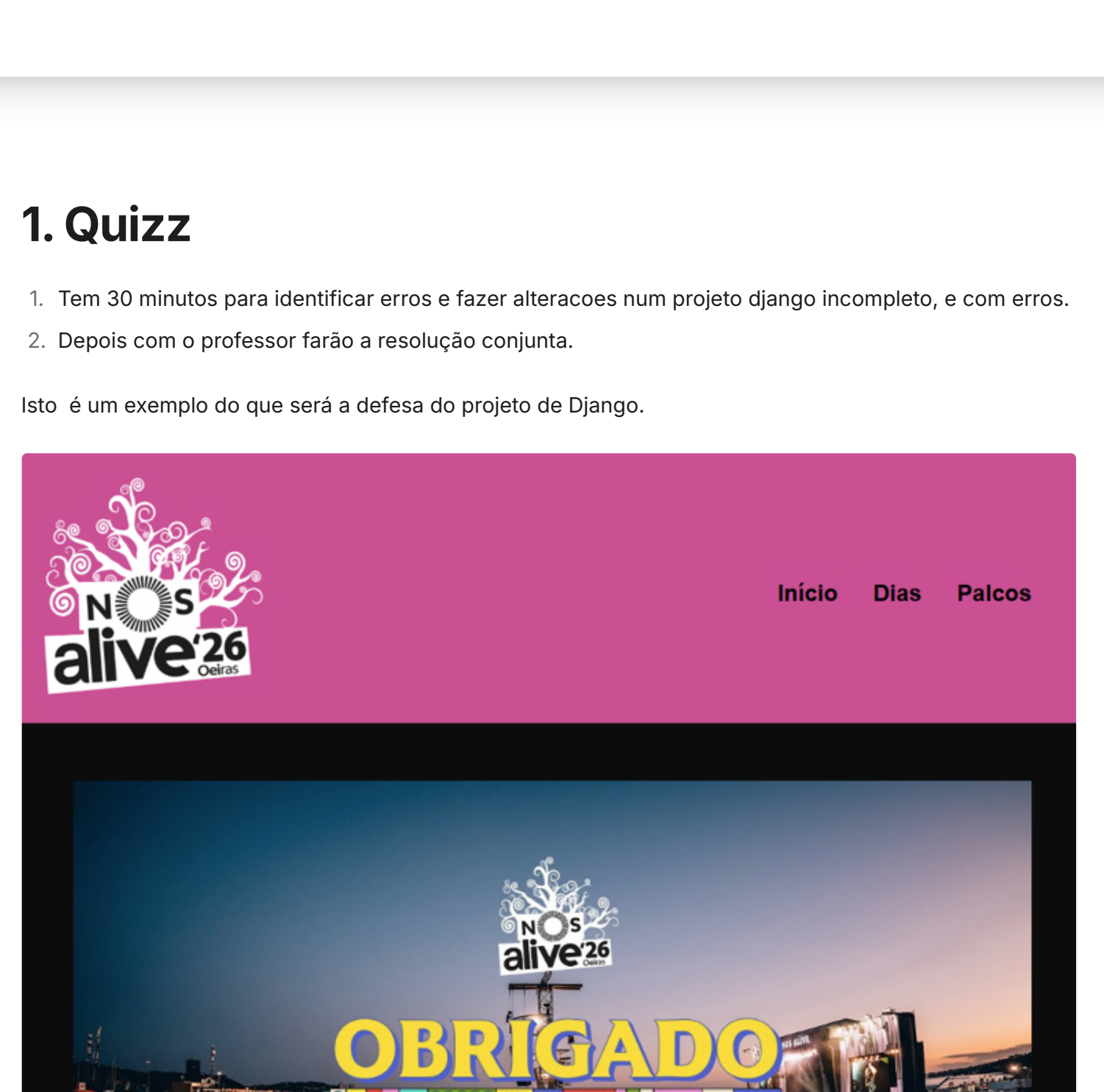
Nesta ficha irá:

1. demonstrar num quizz os conhecimentos adquiridos com a realização da Ficha 7
2. continuar a trabalhar no seu projeto Django.
   a. irá mostrar o seu projeto ao seu professor e registar sugestões de melhoria
   b. irá exercitar a criação de páginas com formulários que permitem operações CRUD sobre váris tabelas da base de dados.
   c. irá criar um video-tutorial sobre o Django
   d. Irá criar uma nova página sobre a sua aplicação

1. Quizz

1. Tem 30 minutos para identificar erros e fazer alteracoes num projeto django incompleto, e com erros.
2. Depois com o professor farão a resolução conjunta.

Isto é um exemplo do que será a defesa do projeto de Django.

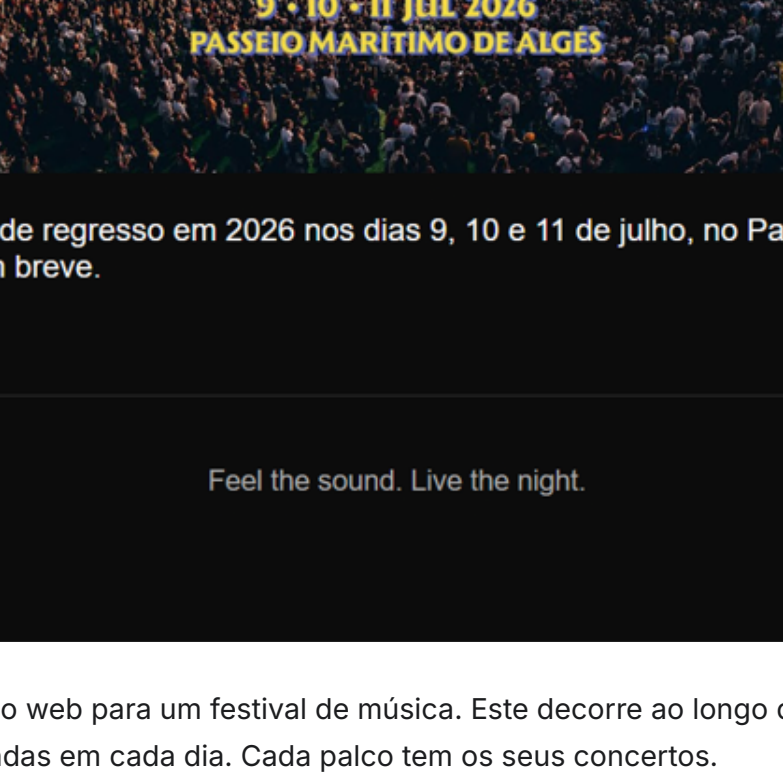


O NOS Alive está de regresso em 2026 nos dias 9, 10 e 11 de julho, no Passeio Marítimo de Algés. Bilhetes em breve.

Feel the sound. Live the night.

Considere uma aplicação web para um festival de música. Este decorre ao longo de vários dias, havendo vários concertos de bandas em cada dia. Cada palco tem os seus concertos.

Eis os interfaces existentes:



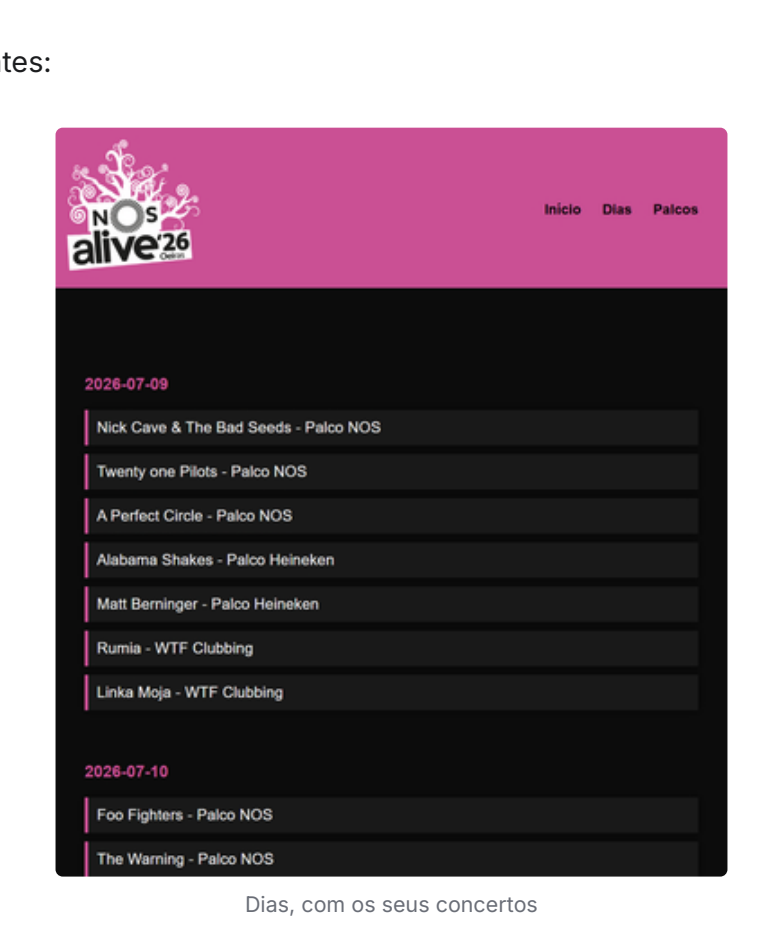
2026-07-09

- Nick Cave & The Bad Seeds - Palco NOS
- Twenty one Pilots - Palco NOS
- A Perfect Circle - Palco NOS
- Alabama Shakes - Palco Heineken
- Matt Berninger - Palco Heineken
- Rumia - WTF Clubbing
- Linka Moja - WTF Clubbing

2026-07-10

- Foo Fighters - Palco NOS
- The Warning - Palco NOS

Dias, com os seus concertos



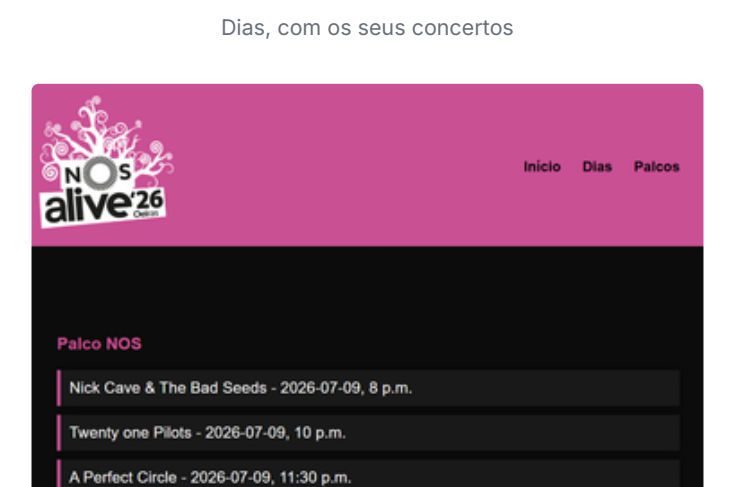
Palco NOS

- Nick Cave & The Bad Seeds - 2026-07-09, 8 p.m.
- Twenty one Pilots - 2026-07-09, 10 p.m.
- A Perfect Circle - 2026-07-09, 11:30 p.m.
- Foo Fighters - 2026-07-10, 8 p.m.
- The Warning - 2026-07-10, 10 p.m.
- Buraka Som Sistema - 2026-07-11, 8 p.m.
- Florence + The Machine - 2026-07-11, 10 p.m.
- Teddy Swims - 2026-07-11, 11:30 p.m.

Palco Heineken

- Alabama Shakes - 2026-07-09, 7 p.m.

Palcos, com os seus concertos



Nick Cave & The Bad Seeds

Dia: 2026-07-09

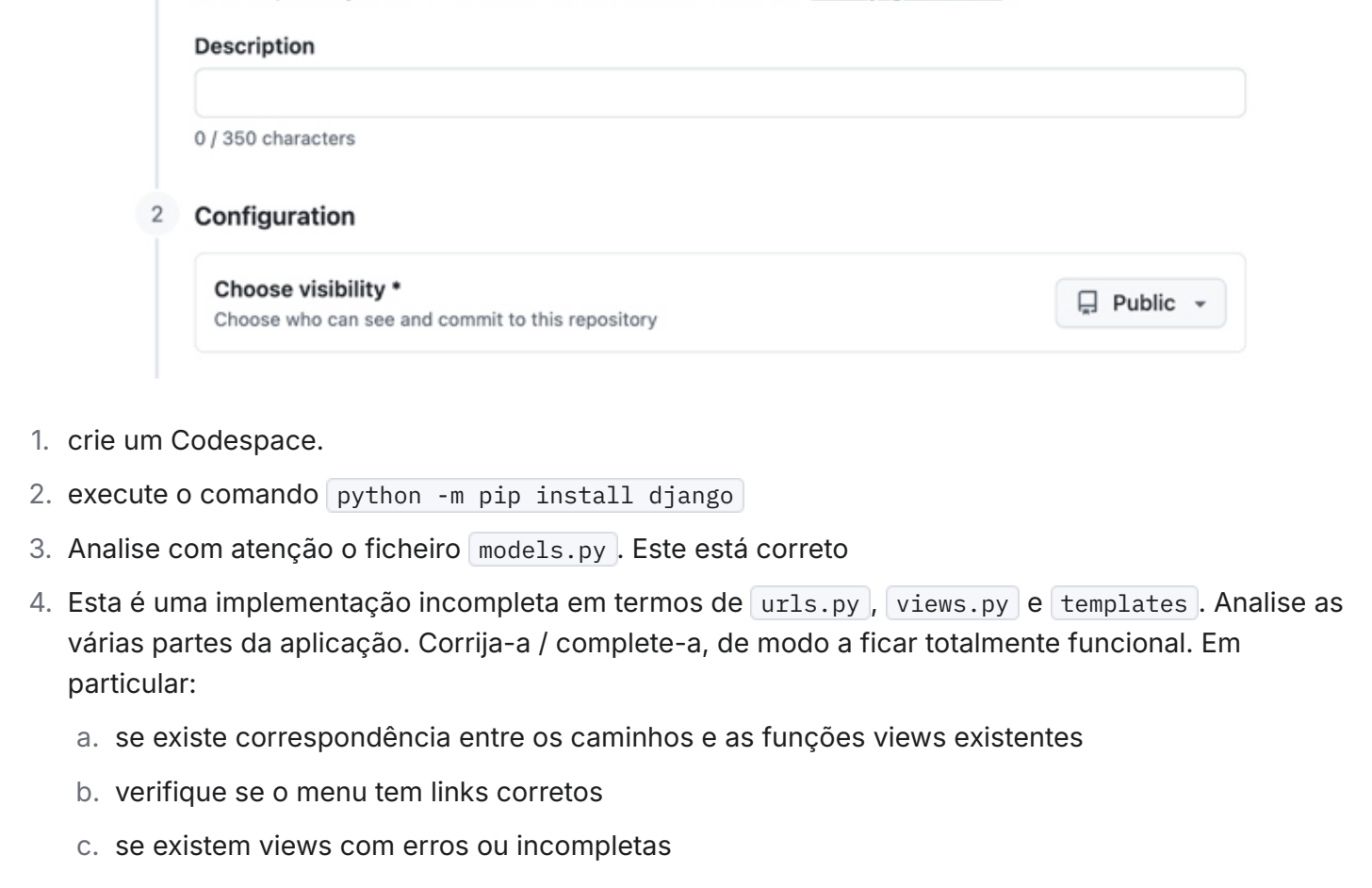
Palco: Palco NOS

Feel the sound. Live the night.

Concerto

Existe um repositório com uma implementação desta aplicação, mas que está incompleta e com alguns erros.

1. Abra este link: https://github.com/ULHT-PW/festival-quizz ↗
2. Clique em "**use this template**"
3. crie um repositorio novo com base neste, e dê um nome festival



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

Start with a template
Templates pre-configure your repository with files.
ULHT-PW/festival-quizz

Include all branches
If enabled, all branches from the template repository will be included.
Off

1 **General**

Owner *
luciostuder

Repository name *
festival

Great repository names are short and memorable. How about cuddly-guacamole?

Description

0 / 350 characters

2 **Configuration**

Choose visibility *
Choose who can see and commit to this repository
Public

1. crie um Codespace.
2. execute o comando `python -m pip install django`
3. Analise com atenção o ficheiro `models.py`. Este está correto
4. Esta é uma implementação incompleta em termos de `urls.py`, `views.py` e `templates`. Analise as várias partes da aplicação. Corrija-a / complete-a, de modo a ficar totalmente funcional. Em particular:
   a. se existe correspondência entre os caminhos e as funções views existentes
   b. verifique se o menu tem links corretos
   c. se existem views com erros ou incompletas
   d. se existem todos os templates
   e. se os templates teem todos os links funcionais

No final, dê commit e submeta o link do seu repo.

2. Portfólio

2.1 Feedback

Peça para falar com o seu docente quando estiver disponível. Enquanto não esteja, passe para a página seguinte (formulários)

Analisem juntos a modelação

Sugestões que lhe sejam feitas, deverá incluir no making-off, descrevendo depois que alterações fez.

É importante que o making-off tenha informação da data e do commit da alteração efectuada. Caso a sua modelação nao inclua isso, complete.

O seu projeto será avaliado pelos commits que realizou, o que foi alterado, e quantidade de iterações. Nesse sentido, encontre maneira de incluir no making off o reporte deste indicador.

2.2 Desvendando o poder dos formulários no seu Portfolio

Relembre o que fizemos na aula teórica, criação de páginas que permitem criar, editar ou apagar elementos usando formulários:

- repositório de código com descrição no README ↗
- Video-tutorial ↗

Já criou páginas que listam o conteúdo de todas as tabelas da sua base de dados (disciplinas, projetos, tecnologias, competências, formações, etc).

1. Projetos

No seu portfolio, crie páginas que permitam gerir os projetos da sua aplicação com operações CRUD:

- criar um novo projeto;
- editar um projeto;
- apagar um projeto.

Em sítios que considera adequados, coloque botões que permitam abrir as respetivas páginas para fazer essas operações CRUD.

2. Restantes

Faça o mesmo para as classes:

- tecnologia
- competência
- formação

Lembre-se de tomar nota e posteriormente documentar esta implementação no making off. Exemplos poderão ser:

- dificuldades encontradas
- erros
- aspectos que considerou importantes
- o que gostou e o que nao gostou
- o que acho difícil de entender
- o que achou útil
- considerações, por exemplo de como o Django se mostra vantajoso na forma de construir uma aplicação web
- etc.

2.4 Página "Sobre esta Aplicação"

Crie uma página "sobre esta aplicação", onde explica aspectos técnicos da sua aplicação. Esta secção irá crescer bastante nas próximas semanas. Para esta semana inclua os seguintes conteúdos:

1. Arquitetura MVT do Django

- Faça em papel e lápis o desenho da arquitetura e insira a sua fotografia na página
- Explique cada um dos seus componentes e cada um dos fluxos de informação.
- inclua num elemento `iframe` o video-tutorial que irá criar de acordo com as indicações da página seguinte, a explicar a criação e funcionamento de uma página com um formulário para criar um novo projeto.

2. Modelação

- Apresente o diagrama entidade relação no desenho que fez.
- Apresente a modelação em models.py
- Pode incluir a seguir uma versão mais completa, recorrendo a algum software (exemplo ↗), ou usando IAGenerativa.

3. Tecnologias

- Apresente as tecnologias que usou para a construção da aplicação Django (Django, HTML, CSS, Git, GitHub etc). Esta lista irá crescer nas próximas semana.
- Para tal use a classe tecnologias, que deverão ser as que estão associadas ao projeto Portfolio de Programação Web. Garanta que tecnologias tem um campo que permite incluir uma descrição sobre o que essa tecnologia faz, o que essa tecnologia permite, e aspectos que gostou ou não dessa tecnologia.
- Estruture-as em tipos: frontend, backend, base de dados, storage, outros (Git, CodeSpaces, etc). Crie uma classe para tipo e inclua nas tecnologias. (documente estas alterações no making of).

4. Estrutura das páginas

- Faça um desenho do mapa da estrutura de navegação da aplicação.

5. Repositório GitHub

- inclua um link para o seu repositório GitHub.
- Descreva como o usou, as vantagens que esta tecnologia traz para. odesenvolvimento de software

6. Making-Of

- Apresente os conteúdos que vai recolhendo do making-Of. Eventualmente crie apenas uma breve apresentação do desafio e em que consiste, e crie uma nova página que mostra todos os detalhes do que foi reunindo nesta tabela.
- Caso tenha apontamentos em markdown relativos ao Making off que queira manter a formatação, pode usar o Markdownify, com ose explica na página seguinte.

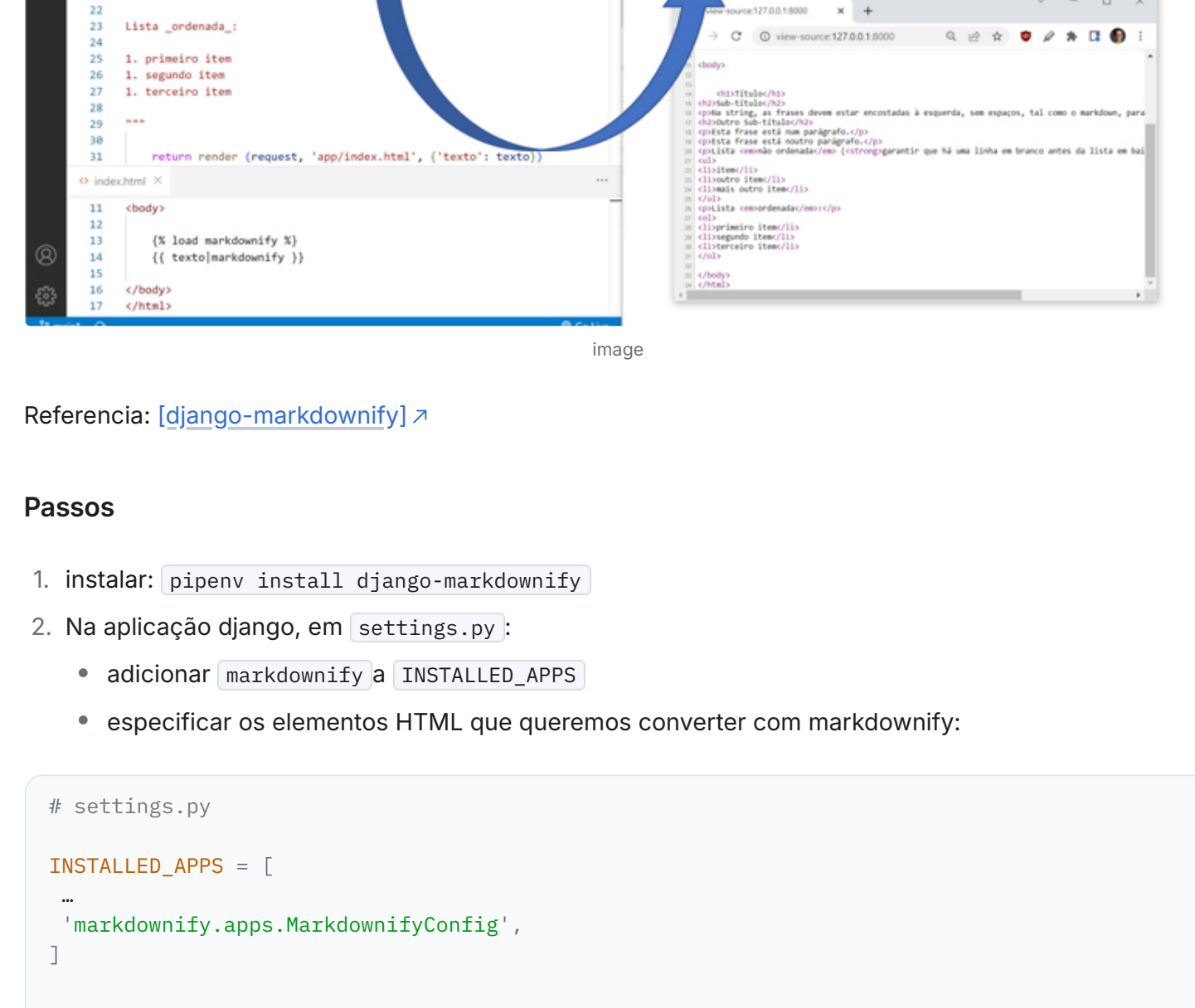
Markdownify

Markdownify é um módulo que permite no django converter markdown em HTML 😎!

Permite por exemplo guardar numa base de dados, num campo do tipo TextField ou CharField, texto com formatação em Markdown. Pode ser útil para apontamentos que tenha feito relativo ao Making off que queira integrar com uma formatação adequada.

Na imagem em baixo:

- a variável `texto` tem texto com marcadores markdown (`#` e `##` para especificar titulos, `*` listar não ordenadamente, `1.` para lista ordenada, `**` para bold, `__` para itálico, ...)
- a variável `texto` é passada como contexto na função `render`, para o template HTML
- no ficheiro HTML a variável é renderizada, usando-se o filtro `markdownify` em `{{ texto|markdownify }}`. Super simples!
- Veja como resulta a conversão em HTML.



```
from django.shortcuts import render

def index_view(request):

    texto = """
# Titulo

## Sub-titulo
Na string, as frases devem estar encostadas à esquerda, sem espaços,
tal como o markdown, para que os marcadores funcionem.

## Outro Sub-titulo

Esta frase está num parágrafo.

Esta frase está noutro parágrafo.

Lista *não ordenada* (**garantir que há uma linha em branco antes da
lista em baixo!**):

* item
* outro item
* mais outro item

Lista _ordenada_:

1. primeiro item
1. segundo item
1. terceiro item

"""

    return render (request, 'app/index.html', {'texto': texto})
```

```
<body>

    {% load markdownify %}
    {{ texto|markdownify }}

</body>
</html>
```

Título

Sub-título

Na string, as frases devem estar encostadas à esquerda, sem espaços, tal como o markdown, para que os marcadores funcionem.

Outro Sub-título

Esta frase está num parágrafo.

Esta frase está noutro parágrafo.

Lista *não ordenada* (**garantir que há uma linha em branco antes da lista em baixo!**):

- item
- outro item
- mais outro item

Lista *ordenada*:

1. primeiro item
2. segundo item
3. terceiro item

image

Referencia: [django-markdownify] ↗

Passos

1. instalar: `pipenv install django-markdownify`
2. Na aplicação django, em `settings.py`:
   - adicionar `markdownify` a `INSTALLED_APPS`
   - especificar os elementos HTML que queremos converter com markdownify:

```
# settings.py

INSTALLED_APPS = [
 …
 'markdownify.apps.MarkdownifyConfig',
]


MARKDOWNIFY = {
   "default": {
      "WHITELIST_TAGS": [
        'a', 'abbr', 'acronym',
        'strong', 'b',
        'blockquote', 'em', 'i',
        'ul', 'li', 'ol',
        'p',
        'h1', 'h2', 'h3', 'h4',
      ]
   },

   "alternative": {
      "WHITELIST_TAGS": ["a", "p"],
      "MARKDOWN_EXTENSIONS": ["markdown.extensions.fenced_code", ]
   }
}
```

3. Na página html onde queremos usar:
   - carregar o filtro
   - utilizar o filtro nas variáveis que usarmos ou passarmos ↗:

```
<!-- pagina.html -->

{% load markdownify %}

<div>
   {{ texto | markdownify }}
</div>
```

2.3 O meu primeiro Video-tutorial

- Pretende-se que crie um video-tutorial onde explica como funciona a sua aplicação em termos funcionais do Django. Esta atividade poderá realizar em casa.
- Veja o videotutorial sobre operações CRUD na aplicação biblioteca ↗ como exemplo de um video-tutorial semelhante ao que irá gravar (insira um comentário ou like, caso considere que foi útil).

Tarefas a realizar:

1. Crie um video-tutorial semelhante a explicar todos os passos para a construção de uma página que permite criar um novo projeto através de um formulário.
2. Explique em detalhe os passos do desenvolvimento. Fale ou faça mesmo "live coding" do que precisa inserir nos ficheiros models.py, forms.py, urls.py, views.py, e template HTML, linguagem template.
3. Depois explique a sequencia de os passos do funcionamento da aplicação:
   a. utilizador no browser clica num link
   b. link está associado a uma rota que tem mapeada uma função view
   c. view interage com BD e renderiza os conteudos num HTML que retorna para o utilizador.
4. Guarde o vídeo no Youtube.

Commit

Faça commit para o GitHub das alterações feitas.

Submeta no Moodle o link para o seu repositório

Only this page

All pages

Powered by GitBook